

Лабораторная работа №14

Архитектура компьютеров

Безходарнова А.В.

25 апреля 2026

Российский университет дружбы народов, Москва, Россия

Докладчик

- Безходарнова Алиса Викторовна
- Студентка НКАбд-01-25
- Алиса
- Российский университет дружбы народов
- 1032253545@rudn.ru



Изучить основы программирования в оболочке ОС UNIX. Научиться писать более сложные командные файлы с использованием логических управляющих конструкций и циклов.

1. Написать командный файл, реализующий упрощённый механизм семафоров. Командный файл должен в течение некоторого времени t_1 дожидаться освобождения ресурса, выдавая об этом сообщение, а дождавшись его освобождения, использовать его в течение некоторого времени $t_2 < t_1$, также выдавая информацию о том, что ресурс используется соответствующим командным файлом (процессом). Запустить командный файл в одном виртуальном терминале в фоновом режиме, перенаправив его вывод в другой (`> /dev/tty#`, где # — номер терминала куда перенаправляется вывод), в котором также запущен этот файл, но не фоновом, а в привилегированном режиме. Доработать программу так, чтобы имелась возможность взаимодействия трёх и более процессов.
2. Реализовать команду `man` с помощью командного файла. Изучите содержимое каталога `/usr/share/man/man1`. В нем находятся архивы текстовых файлов, содержащих справку по большинству установленных в системе программ и команд. Каждый архив можно открыть командой `less` сразу же просмотрев

Командный процессор (командная оболочка, интерпретатор команд shell) — это программа, позволяющая пользователю взаимодействовать с операционной системой компьютера. В операционных системах типа UNIX/Linux наиболее часто используются следующие реализации командных оболочек: – оболочка Борна (Bourne shell или sh) — стандартная командная оболочка UNIX/Linux, содержащая базовый, но при этом полный набор функций; – С-оболочка (или csh) — надстройка на оболочкой Борна, использующая С-подобный синтаксис команд с возможностью сохранения истории выполнения команд; – оболочка Корна (или ksh) — напоминает оболочку С, но операторы управления программой совместимы с операторами оболочки Борна; – BASH — сокращение от Bourne Again Shell (опять оболочка Борна), в основе своей совмещает свойства оболочек С и Корна (разработка компании Free Software Foundation). POSIX (Portable Operating System Interface for Computer Environments) — набор стандартов описания интерфейсов взаимодействия операционной системы и прикладных программ. Стандарты POSIX разработаны комитетом IEEE (Institute of

Создаю файл

```
[avbezkhodarnova1@fedora presentation]$ cd  
[avbezkhodarnova1@fedora ~]$ touch /tmp/semaphore  
[avbezkhodarnova1@fedora ~]$ nano semaphore.sh
```

Рис. 1: Файл первый

Пишу программу

```
GNU nano 7.2 semaphore.sh
#!/bin/bash
SEM="/tmp/semaphore"
t1=5
t2=3
while ! ln $SEM $SEM.lock 2>/dev/null; do
    echo "Жду ресурс $t1 сек" > /dev/tty2
    sleep $t1
done
echo "Использую ресурс $t2 сек" > /dev/tty2
sleep $t2
rm $SEM.lock
```

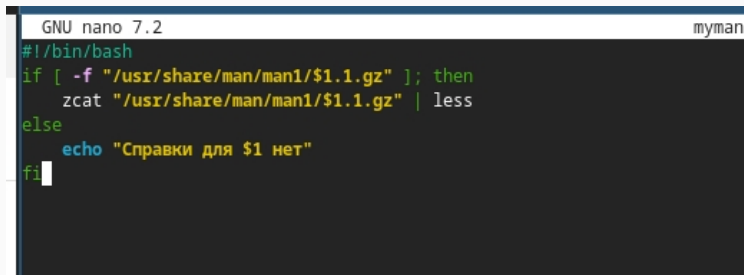
Рис. 2: Командный файл

Делаю файл исполняемым

```
[avbezkhodarnova1@fedora ~]$ chmod +x semaphore.sh  
[avbezkhodarnova1@fedora ~]$ ./semaphore.sh > /dev/tty2 &  
[1] 2849  
[avbezkhodarnova1@fedora ~]$
```

Рис. 3: Запуск файла

Создаю второй программный код

A screenshot of a terminal window with a dark background. The title bar at the top shows "GNU nano 7.2" on the left and "myman" on the right. The terminal contains a shell script with the following lines: a shebang line "#!/bin/bash", an if statement "if [-f "/usr/share/man/man1/\$1.1.gz"]; then", an indented command "zcat "/usr/share/man/man1/\$1.1.gz" | less", an else branch "else", an indented command "echo "Справки для \$1 нет"", and a closing "fi" statement. A white cursor is positioned at the end of the "fi" line.

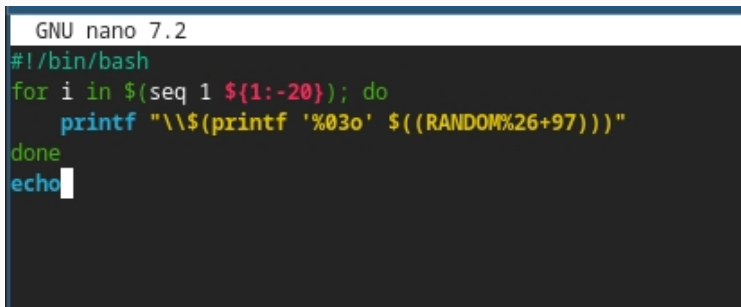
```
GNU nano 7.2 myman
#!/bin/bash
if [ -f "/usr/share/man/man1/$1.1.gz" ]; then
    zcat "/usr/share/man/man1/$1.1.gz" | less
else
    echo "Справки для $1 нет"
fi
```

Рис. 4: Второй код

Запускаю его и проверяю работу

```
.\" DO NOT MODIFY THIS FILE! It was generated by help2man 1.48.5.
.TH LS "1" "September 2024" "GNU coreutils 9.3" "User Commands"
.SH NAME
ls \- list directory contents
.SH SYNOPSIS
.B ls
[\fI\,OPTION\|\fR]... [\fI\,FILE\|\fR]...
.SH DESCRIPTION
.\" Add any additional description here
.PP
List information about the FILES (the current directory by default).
Sort entries alphabetically if none of \fB\-\cftuvSUX\|fR nor \fB\-\sort\|fR is specified.
.PP
Mandatory arguments to long options are mandatory for short options too.
.TP
\fB\-a\|fR, \fB\-\all\|fR
do not ignore entries starting with .
.TP
\fB\-A\|fR, \fB\-\almost\-\all\|fR
do not list implied . and ..
.TP
\fB\-\author\|fR
with \fB\-l\|fR, print the author of each file
.TP
\fB\-b\|fR, \fB\-\escape\|fR
print C\-\style escapes for nongraphic characters
.TP
\fB\-\block\-\size\|fR=\fI\,SIZE\|\fR
with \fB\-l\|fR, scale sizes by SIZE when printing them;
e.g., '\fB\-\block\-\size=M\|fR'; see SIZE format below
.TP
\fB\-B\|fR, \fB\-\ignore\-\backups\|fR
do not list implied entries ending with ~
.TP
\fB\-c\|fR
with \fB\-lt\|fR: sort by, and show, ctime (time of last
change of file status information);
with \fB\-l\|fR: show ctime and sort by name;
otherwise: sort by ctime, newest first
.TP
\fB\-C\|fR
list entries by columns
.TP
\fB\-\color\|fR[=\fI\,WHEN\|\fR]
color the output WHEN; more info below
.TP
\fB\-d\|fR, \fB\-\directory\|fR
```

Также пишу последний программный код

A screenshot of a terminal window with a dark background and a light blue title bar that reads "GNU nano 7.2". The terminal contains a shell script with the following lines:

```
#!/bin/bash
for i in $(seq 1 ${1:-20}); do
    printf "\\$(printf '%03o' $((RANDOM%26+97)))"
done
echo
```

 The script uses color-coded syntax: blue for the shebang, green for the loop structure, red for the range in the seq command, and yellow for the printf command. A white cursor is positioned at the end of the last line.

Рис. 6: Третий код

Запускаю код, проверяя его работу

```
[avbezkhodarnova1@fedora ~]$ chmod +x random.sh  
[avbezkhodarnova1@fedora ~]$ ./random.sh 30  
axblkobptvomerfukciyswanghjoob  
[avbezkhodarnova1@fedora ~]$
```

Рис. 7: Запуск кода

В ходе данной лабораторной работы я изучила основы программирования в оболочке ОС UNIX. Научилась писать более сложные командные файлы с использованием логических управляющих конструкций и файлов

1. Найдите синтаксическую ошибку в следующей строке: Там нужно поставить пробелы
2. Как объединить (конкатенация) несколько строк в одну? Просто написать переменные подряд
3. Найдите информацию об утилите seq. Какими иными способами можно реализовать её функционал при программировании на bash? seq генерирует числа; замену можно сделать через for

4. Какой результат даст вычисление выражения $\$((10/3))$? 3 Целочисленное деление
5. Укажите кратко основные отличия командной оболочки zsh от bash. zsh имеет автодополнение, исправление опечаток и расширенные glob-паттерны
6. Проверьте, верен ли синтаксис данной конструкции `1 for ((a=1; a <= LIMIT; a++))`
Синтаксис верен, работает в bash

7. Сравните язык `bash` с какими-либо языками программирования. Какие преимущества у `bash` по сравнению с ними? Какие недостатки? Преимущества: работа с процессами и файлами напрямую; недостатки: медленный, плох для математики и сложных структур

